

Experiment-1

WRITE A PROGRAM TO ANALYZE AND COMPARE THE TIME COMPLEXITY OF BASIC OPERATION SEARCHING AN ELEMENT IN ARRAYS AND LINKED LISTS

```
#include <stdio.h>

#include <stdlib.h>

#include <time.h>

// Structure for a Node in the Linked List

typedef struct Node

{

int data;

struct Node* next;

} Node;

// Function to perform Linear Search on an Array

int searchArray(int arr[], int size, int target)

{

for (int i = 0; i < size; i++)

{

if (arr[i] == target)

{ return i; // Target found

}

}

return -1; // Target not found

}
```

```

// Function to perform Search on a Linked List int
searchLinkedList(Node* head, int target)

{

Node* current = head;

int index = 0;

while (current != NULL)

{

    if (current->data == target)

{ return index; // Target found

}

current = current->next;

index++;

} return -1; // Target not found

}

// Helper function to insert a node at the end of the Linked List
void insertEnd(Node** head, int data)

{

    Node* newNode = (Node*)malloc(sizeof(Node));

newNode->data = data;

newNode->next = NULL;

if (*head == NULL) {

    *head = newNode;

return;

} Node* temp = *head;

```

```

    while (temp->next != NULL)

    {

temp = temp->next;

    }

temp->next = newNode;

    }

    int main()

    {

int size = 50000; // Dataset size

    int target = 49999; // Value near the end to force a Worst-Case
scenario

// Allocate memory for the array

int* array = (int*)malloc(size * sizeof(int));

    // Initialize the Linked List

Node* head = NULL;

// Populate both data structures with the same sequential data

for (int i = 0; i < size; i++) {

    array[i] = i;

insertEnd(&head, i);

} printf("Dataset Size: %d\n", size);

printf("Searching for target: %d (Worst-Case Analysis)\n\n",
target); // --- Measure Array Search Time ---

clock_t startArray = clock();

    int arrayResult = searchArray(array, size, target);

```

```

clock_t endArray = clock();

double timeArray = ((double)(endArray - startArray)) /
CLOCKS_PER_SEC;

printf("Array Search Result Index: %d\n", arrayResult);
printf("Time taken by Array Search: %f seconds\n\n", timeArray);
// --- Measure Linked List Search Time ---

clock_t startLL = clock();

int llResult = searchLinkedList(head, target);

clock_t endLL = clock();

double timeLL = ((double)(endLL - startLL)) / CLOCKS_PER_SEC;
printf("Linked List Search Result Index: %d\n", llResult);
printf("Time taken by Linked List Search: %f seconds\n\n",
timeLL);

// --- Analysis Summary ---

if (timeArray < timeLL) { printf("Conclusion: Array search was
faster.\n");

    printf("Why? Arrays benefit from CPU cache locality (contiguous
memory blocks).\n"); }

else

{ printf("Conclusion: Linked List search was faster or
equivalent.\n"); }

// Clean up allocated memory free(array); Node* current = head;
while (current != NULL)

{ Node* nextNode = current->next;

free(current);

current = nextNode;

} return 0;

```

```
}
```

OUTPUT

```
Dataset Size: 50000 Searching for target: 49999 (Worst-Case Analysis)
Array Search Result Index: 49999 Time taken by Array Search: 0.000102
seconds Linked List Search Result Index: 49999 Time taken by Linked
List Search: 0.000159 seconds Conclusion: Array search was faster. Why?
Arrays benefit from CPU cache locality (contiguous memory blocks).
```

Experiment-2

Write A Program To Analyze And Compare The Time Complexity Of Basic Operations Insertion On Arrays And Linked Lists Using C

```
#include <stdio.h>

#include <stdlib.h>

#include <time.h>

#define MAX_SIZE 100000

// Define Linked List Node

Structure typedef struct Node { int data; struct Node* next; }
Node;

// --- ARRAY OPERATIONS ---

// Insert at the beginning of an array: O(N) due to element
shifting

void insertArrayAtBeginning(int arr[], int *size, int value)

{
```